JDBC (Java Database Connectivity)

Step 1: Install Software

You need three things.

1. Java JDK

You already have JDK installed.
Check:

```
java -version
```

2. MySQL

Install MySQL.
During installation remember

```
Username : root
Password : root
```

(or whatever password you choose.)

3. VS Code

Install

- VS Code
- Java Extension Pack

Step 2: Download MySQL Connector/J

Download
mysql-connector-j-26.7.0.jar
This is called the **JDBC Driver**.
Without it Java cannot understand how to communicate with MySQL.
Move it into your project.
Example

```
JdbcProject

    src

        JdbcDemo.java

    lib

        mysql-connector-j-26.7.0.jar
```

Step 3: Create Java Project

In VS Code

```
Ctrl+Shift+P
```

Type

```
Java: Create Java Project
```

Choose

```
No Build Tools
```

Name

```
JdbcProject
```

Project becomes

```
JdbcProject

    src

    lib
```

Copy the JDBC jar into

```
lib
```

Step 4: Focus on Java Projects View

Open

```
View

→ Command Palette

→ Java: Focus on Java Projects View
```

You will see

```
JAVA PROJECTS

JdbcProject

    Referenced Libraries

    JDK

    src
```

What is Referenced Libraries?

Java automatically loads external libraries from here.
It contains

```
JDK

+

your jar files
```

Example

```
Referenced Libraries

    mysql-connector-j-26.7.0.jar
```

If it isn't there
Right Click

```
Referenced Libraries

Add Library
```

or

```
Add JAR
```

Choose

```
mysql-connector-j-26.7.0.jar
```

Now Java can access

```
com.mysql.cj.jdbc.Driver
```

Step 5: Create Database

Open MySQL
Execute

```
CREATE DATABASE college;
```

Now

```
USE college;
```

Create table

```
CREATE TABLE student(

    id INT PRIMARY KEY,

    name VARCHAR(30),

    branch VARCHAR(20)

);
```

Verify

```
SHOW TABLES;
```

Output

```
student
```

Check structure

```
DESC student;
```

Output

```
id
name
branch
```

Step 6: Java Program

```
import java.sql.*;
```

Why?
Everything related to JDBC is inside

```
java.sql
```

It imports

```
Connection

Statement

PreparedStatement

DriverManager

SQLException

ResultSet
```

Main Method

```
public static void main(String args[]) throws Exception
```

Program starts here.

Step 7: Load Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

What is this?
It loads

```
MySQL Driver
```

into JVM memory.
Think of it as

```
Java
   ↓

Loads Driver

↓

Driver understands MySQL protocol

↓

Can communicate with MySQL Server
```

Without driver

```
Java
↓

Cannot understand MySQL
```

Older JDBC required this line.
Modern JDBC often loads the driver automatically if the JAR is present, but keeping it is fine and improves compatibility.

Step 8: Make Connection

```
Connection con =DriverManager.getConnection(

"jdbc:mysql://localhost:3306/college",

"root",

"root"

);
```

Here

```
jdbc:mysql://
```

means
Use JDBC with MySQL.

```
localhost
```

means
Database is on your own computer.

```
3306
```

Default MySQL Port.

```
college
```

Database name.

```
root
```

Username.

```
root
```

Password.
After success

```
Connection

↓

Open

↓

Ready
```

Step 9: Create PreparedStatement

```
PreparedStatement ps =con.prepareStatement(

"insert into student values(?,?,?)"

);
```

Question marks are placeholders.

```
?

?

?
```

Instead of writing

```
insert into student values(101,'Rahul','CSE')
```

we use

```
?

?

?
```

Step 10: Set Values

```
ps.setInt(1,101);
```

Meaning

```
First ?

↓

101
```

```
ps.setString(2,"Rahul");
```

Second

```
?

↓

Rahul
```

```
ps.setString(3,"CSE");
```

Third

```
?

↓

CSE
```

Now query becomes

```
insert into student
values(101,'Rahul','CSE');
```

Step 11: Execute

```
ps.executeUpdate();
```

Used for

```
INSERT

UPDATE

DELETE

CREATE

DROP
```

Returns

```
Number of rows affected
```

Example

```
1
```

Step 12: Print

```
System.out.println("Record Inserted");
```

Output

```
Record Inserted
```

Step 13: Create Statement

```
Statement st =con.createStatement();
```

Used for simple SQL without parameters.
Example

```
SELECT *
```

Step 14: Execute Query

```
ResultSet rs =st.executeQuery(

"select * from student"

);
```

executeQuery() is used only for SELECT statements.
The returned ResultSet is like a cursor that points to the rows returned by the database.

Step 15: Read ResultSet

```
while(rs.next())
```

Initially

```
Cursor

↓

Before First Row
```

```
ID Name

101 Rahul
102 Amit
103 Neha
```

rs.next()
↓
Moves to

```
101 Rahul
```

Again
↓
Moves

```
102 Amit
```

Again
↓
Moves

```
103 Neha
```

Again
↓
No row
↓
Returns false
Loop ends.

Step 16: Get Values

```
rs.getInt(1)
```

Gets first column

```
ID
```

```
rs.getString(2)
```

Gets second column

```
Name
```

```
rs.getString(3)
```

Gets third column

```
Branch
```

Instead of column numbers, you can also use column names:

```
rs.getInt("id");
rs.getString("name");
rs.getString("branch");
```

This is often easier to read and remains correct even if the column order changes.

Step 17: Output

Suppose table contains

```
101 Rahul CSE

102 Amit IT

103 Neha ECE
```

Output

```
Record Inserted

101 Rahul CSE

102 Amit IT

103 Neha ECE
```

Step 18: Close Connection

```
con.close();
```

Always close database resources to free memory and database connections. A more robust approach is to use *try-with-resources*, which closes Connection, PreparedStatement, Statement, and ResultSet automatically.

Complete Flow Diagram

```
Java Program

       |

       ▼

Load JDBC Driver

       |

       ▼

DriverManager

       |

       ▼

Connect to MySQL

       |

       ▼

Connection Object

       |

       ▼

Prepare SQL Query

       |

       ▼

Bind Parameters

       |

       ▼

Execute INSERT

       |

       ▼

Record Stored in Database

       |

       ▼

Execute SELECT

       |

       ▼

ResultSet Returned

       |

       ▼

Read Rows One by One

       |

       ▼

Display Output

       |

       ▼

Close Connection
```

Common Errors and Fixes

Error	Cause	Solution
ClassNotFoundException: com.mysql.cj.jdbc.Driver	MySQL Connector/J JAR not added	Add the JAR to **Referenced Libraries** Java Project
Communications link failure	MySQL Server is not running	Start the MySQL service
Access denied for user 'root'	Incorrect username or password	Use the correct MySQL credentials
Unknown database 'college'	Database not created	Run CREATE DATABASE college;
Table 'student' doesn't exist	Table missing	Run the CREATE TABLE student statem
Duplicate entry '101' for key 'PRIMARY'	id already exists	Use a different ID or change the SQL duplicates